

ANÁLISE DE ENGENHARIA REVERSA

Aplicativo Mobile To-Do

Expo SDK 57 • React Native 0.86 • TypeScript
Expo Router • expo-sqlite

Respostas verificadas com base no código-fonte e no BUILD_LOG.md do projeto.

Documento técnico • versão em português

Visão geral

Este documento reúne as respostas às principais questões de engenharia reversa do aplicativo **Mobile To-Do**. O objetivo é explicar, de forma rastreável, como o projeto está organizado, como os dados são persistidos, como uma operação percorre as camadas da aplicação e quais decisões de arquitetura foram tomadas durante o desenvolvimento.

Stack principal: Expo SDK 57, React Native 0.86, TypeScript, Expo Router e expo-sqlite.

Sumário

Nº	Seção
01	Estrutura do projeto
02	Arquitetura e estado
03	Persistência com SQLite
04	Fluxo completo: criação de uma tarefa
05	Navegação
06	Notificações
07	Decisões tomadas pelo agente
08	Análise do BUILD_LOG

1. Estrutura do projeto

O aplicativo utiliza uma **estrutura de pastas em camadas** dentro de `src/`. As rotas do Expo Router ficam concentradas em `src/app/`, enquanto as demais responsabilidades permanecem fora do diretório de rotas.

Parte	Localização e responsabilidade
UI / telas	<code>src/app/index.tsx</code> (lista de tarefas), <code>src/app/task/[id].tsx</code> (editor — <code>/task/new</code> cria e <code>/task/<id></code> edita) e <code>src/app/categories.tsx</code> (categorias). Componentes reutilizáveis ficam em <code>src/components/</code> .
Modelos de dados	<code>src/models/task.ts</code> e <code>src/models/category.ts</code> . Os tipos <code>Row</code> representam o formato bruto do SQLite; os tipos de domínio usam booleanos.
SQLite / banco	<code>src/db/database.ts</code> e <code>src/db/migrations.ts</code> . As consultas ficam em <code>taskRepository.ts</code> e <code>categoryRepository.ts</code> .
Navegação	Expo Router, com as rotas definidas pelos arquivos de <code>src/app/</code> e registradas no <code><Stack></code> de <code>_layout.tsx</code> .
Notificações	<code>src/services/notificationService.ts</code> , responsável por configurar, agendar e cancelar lembretes.
Estado compartilhado	<code>src/state/AppDataContext.tsx</code> , que fornece a versão de atualização e mensagens temporárias.
Utilitários	<code>src/utils/date.ts</code> para datas e <code>src/utils/error.ts</code> para mensagens de erro.

Organização em resumo: `_layout.tsx` compõe os providers (**ThemeProvider** → **SQLiteProvider** → **AppDataProvider** → **Stack**). As telas leem e gravam por meio dos repositories; os repositories são a única camada que conversa diretamente com SQL. Comportamentos específicos da plataforma, como notificações, ficam em `services/`, enquanto funções puras ficam em `utils/`.

2. Arquitetura e estado

A fonte de verdade é o banco SQLite. O estado do React funciona como um cache local dos resultados das consultas, mantido em cada tela (`tasks`, `categories`, `loading`, `error` e filtros).

O único estado compartilhado é o `AppDataContext`, que mantém:

- `version` — contador incrementado por `refresh()` após cada gravação.
- `notice` — mensagem temporária, como aviso de permissão negada, exibida na lista.

Como a interface é atualizada:

- A tela que grava executa o repository correspondente (`createTask`, `updateTask` etc.).
- Depois chama `notifyRefresh()`, isto é, `refresh()` do contexto, incrementando `version`.
- As telas de leitura usam `useAppDataEffect(reload)`, uma pequena camada sobre `useFocusEffect`. A mudança de `version` altera a identidade do callback e força nova leitura quando a tela está focada.
- `reload()` executa novamente `listTasks` ou `listCategories`, atualiza o estado e provoca o novo render da `FlatList`.

Exemplo: criar uma tarefa segue o fluxo `createTask` → `syncTaskNotification` → `notifyRefresh` → `router.back` → `reload` → `setTasks`.

Padrão reconhecível: repository pattern para isolar SQL, combinado com um pequeno estado global baseado em versão/evento. Redux, Zustand e EventEmitter foram considerados e descartados, conforme registrado na Fase 7 do `BUILD_LOG.md`.

3. Persistência com SQLite

Criação e abertura do banco: `src/app/_layout.tsx:21`.

```
<SQLiteProvider databaseName={DATABASE_NAME} onInit={onInitDatabase}>
```

O nome do banco é `todoapp3.db`. A função `onInitDatabase` é executada a cada abertura e configura WAL, chaves estrangeiras e as migrações:

```
await db.execAsync('PRAGMA journal_mode = WAL;');
await db.execAsync('PRAGMA foreign_keys = ON;');
await runMigrations(db);
```

Schema e migrações: `src/db/migrations.ts` mantém uma lista ordenada de migrações com `{ version, migrate }`. Elas são aplicadas dentro de `withTransactionAsync` e controladas por `PRAGMA user_version`.

```
categories(id, name TEXT NOT NULL UNIQUE COLLATE NOCASE, color)

tasks(id, title, description, completed INTEGER DEFAULT 0,
      dueDateTime, createdAt,
      categoryId INTEGER REFERENCES categories(id) ON DELETE SET NULL,
      notificationId)
```

As datas são armazenadas como **texto ISO 8601 em UTC**. O campo `completed` usa 0/1 no SQLite e é convertido para booleano por `toTask()`. A categoria é uma chave estrangeira anulável; ao excluir uma categoria, `ON DELETE SET NULL` deixa as tarefas sem categoria.

Entidade	Arquivo	Principais funções
Tarefas	<code>taskRepository.ts</code>	<code>listTasks</code> , <code>getTaskById</code> , <code>createTask</code> , <code>updateTask</code> , <code>deleteTask</code> , <code>setTaskNotificationId</code>
Categorias	<code>categoryRepository.ts</code>	<code>listCategories</code> , <code>getCategoryById</code> , <code>createCategory</code> , <code>renameCategory</code> , <code>deleteCategory</code> , <code>countTasksPerCategory</code>

Segurança das consultas: todas as instruções usam parâmetros posicionais `?`; não há interpolação direta de entrada do usuário no SQL.

4. Seguindo uma operação: criação de uma nova tarefa

- 1** Toque em “+ Nova tarefa”. Na lista, `router.push('/task/new')` abre o editor.
- 2** Toque em “Salvar”. `handleSave` valida o título, normaliza a data, descarta uma categoria inexistente e constrói o `TaskInput`.
- 3** **Persistência.** `createTask(db, input)` executa um `INSERT` com parâmetros vinculados e depois relê a linha recém-criada com `getTaskById`.
- 4** **Agendamento do lembrete.** Se existir uma data válida e futura, `syncTaskNotification` cancela um lembrete anterior, verifica permissão, cria o canal Android quando necessário e agenda a notificação. O ID retornado é salvo na tarefa.
- 5** **Atualização da lista.** `notifyRefresh()` incrementa `version`; depois `router.back()` retorna à lista, que executa `reload()` e renderiza a nova tarefa.

Sequência completa: `handleSave` → `createTask (SQLite)` → `syncTaskNotification (lembrete)` → `notifyRefresh (contexto)` → `reload` → `setTasks (UI)`.

5. Navegação

Framework: Expo Router, com navegação baseada em arquivos. As rotas estão em `src/app/` e são expostas pelo `<Stack>` em `_layout.tsx`.

Lista → editor:

```
// Editar uma tarefa existente
router.push({
  pathname: '/task/[id]',
  params: { id: String(task.id) }
});

// Criar uma nova tarefa
router.push('/task/new');
```

O que é transmitido é somente o **ID da tarefa como string**, além do valor especial new para criação. O editor recupera o ID com `useLocalSearchParams`, verifica `isNew = id === 'new'` e, no modo de edição, consulta novamente o SQLite.

Por que somente o ID? Passar o objeto inteiro pela rota poderia levar a uma cópia desatualizada se outra tela modificasse o registro. Usar estado compartilhado entre telas também aumentaria o acoplamento. O ID mantém o banco como fonte única de verdade.

6. Notificações

Toda a lógica de lembretes está centralizada em `src/services/notificationService.ts`.

Como uma notificação é agendada

- `configureNotifications()` é executada uma vez ao carregar `_layout.tsx` e configura o handler para que notificações recebidas em primeiro plano também sejam exibidas.
- No Android, o canal `task-reminders` é criado com alta importância.
- O agendamento utiliza `scheduleNotificationAsync` com trigger do tipo `DATE`, título `Task reminder: <título>`, horário local no corpo e `taskId` nos dados.
- A permissão é solicitada somente durante uma ação do usuário, como salvar uma tarefa com data futura ou concluir/reabrir uma tarefa. A negação não interrompe o salvamento.

Como a notificação é associada à tarefa

O ID retornado por `scheduleNotificationAsync` é salvo na coluna `tasks.notificationId` por `setTaskNotificationId`.

Mudança da data

- Ao editar a data, o serviço primeiro cancela o agendamento antigo e limpa o ID.
- Se a nova data for válida, futura e a tarefa estiver pendente, um novo lembrete é agendado e seu ID é salvo.

Conclusão, exclusão e reabertura

- **Concluída:** cancela o lembrete e define `notificationId = null`.
- **Excluída:** o lembrete é cancelado antes do `deleteTask`.
- **Data removida ou passada:** cancela o lembrete e não agenda outro.
- **Reaberta com data futura:** agenda novamente.
- **Web:** o agendamento é intencionalmente ignorado, pois notificações locais agendadas não são suportadas nesse ambiente; isso é registrado como limitação conhecida.

7. Decisões tomadas pelo agente

1. Navegação por ID

A estratégia escolhida foi `/task/new` para criação e `/task/[id]` para edição. O editor consulta o SQLite novamente. O objeto completo foi rejeitado para evitar dados obsoletos e o estado compartilhado foi considerado desnecessário.

2. SQLite + contador de versão

O SQLite permanece como fonte de verdade. O `AppDataContext` usa `refresh()` para incrementar `version`, fazendo as telas recarregarem quando necessário. `Redux`, `Zustand`, `EventEmitter` e `listeners` diretos do banco foram descartados por adicionarem complexidade.

3. Exclusão de categorias

Foi mantido `ON DELETE SET NULL`. Antes da confirmação, a interface informa quantas tarefas perderão a categoria, usando uma única consulta agrupada em `countTasksPerCategory`.

4. Filtros no SQL

Os filtros são aplicados pelo próprio banco, usando `WHERE` parametrizado e um `ORDER BY` compartilhado. Assim, somente os registros necessários são transferidos para a interface.

5. Arquitetura em camadas

A separação entre telas, `repositories`, banco, serviços, estado e utilitários impede que SQL, APIs de plataforma e UI fiquem misturados e facilita a manutenção.

8. Análise do BUILD_LOG.md

Problema encontrado: durante a Fase 4, `npx expo export --platform web` falhou porque o bundle não estava configurado para incluir o arquivo WASM utilizado pelo `expo-sqlite`.

Tentativa e correção: em vez de testar soluções aleatórias, foram consultadas as orientações atuais do Expo SDK 57 para `expo-sqlite` na web. A solução aplicada foi um novo `metro.config.js`, adicionando `wasm` aos `assets` e configurando os cabeçalhos COEP/COOP necessários para `SharedArrayBuffer`.

Resultado: a exportação web passou após a alteração e o bundle passou a gerar o worker correspondente. Como não havia emulador/AVD disponível, a migração SQL foi validada também com a CLI `sqlite3`.

```
config.resolver.assetExts.push('wasm');

// Middleware com os cabeçalhos necessários:
// Cross-Origin-Embedder-Policy
// Cross-Origin-Opener-Policy
```

O `metro.config.js` tornou-se parte permanente do projeto e a limitação do ambiente foi registrada como **Needs testing**, em vez de ser ignorada.

O log também documenta um segundo caso importante: na Fase 7, o ESLint considerou `version` uma dependência desnecessária. Removê-la, porém, quebraria a atualização automática. Por isso, a dependência foi mantida com um `eslint-disable-line` e comentário explicativo.

O que o BUILD_LOG revela que o código final não revela

- **Por que determinado código existe:** o log conecta o problema, a hipótese, a solução e a verificação.
- **Alternativas rejeitadas:** `Zustand/Redux`, `EventEmitter`, `addDatabaseChangeListener`, filtros em memória, modais e passagem de objetos completos pelas rotas foram avaliados e descartados.
- **Correções e cronologia:** o log registra alterações como a substituição de `countTasksInCategory` por `countTasksPerCategory` e a remoção de uma entrada duplicada.
- **O que ainda não foi verificado:** cada etapa termina com `Current Status: Needs testing`. Entrega de notificações, negação de permissões e persistência após reinício ainda dependem de testes manuais descritos no `TEST_PLAN.md`.
- **Decisões humanas versus decisões do agente:** escolhas como não realizar commits, pular a Fase 15, tratar `SPEC.md` como fonte de verdade e seguir o ritmo por fases aparecem no log, mas não são visíveis no código.

Conclusão

O projeto apresenta uma arquitetura enxuta e bem separada: SQLite como fonte de verdade, repositories para acesso a dados, Expo Router para navegação, um contexto simples para sinalizar atualizações e um serviço isolado para notificações. O BUILD_LOG complementa o código ao registrar problemas, decisões, correções e aquilo que ainda precisa de validação prática.